

Skopeo — Manual End-to-End Testing Guide

A printable runbook for testing the full stack (PostgreSQL + Ktor API + React web UI) with all three auth methods — **Google, email/password, and Facebook** — for both **sign-up** and **login**. **Part 1** runs everything locally with Docker; **Part 2** covers deploying the three components to GCP. **Appendix A** has Firebase project-setup steps for reference (you do **not** need them — the project already exists).

Tip: to make a PDF, open this file in your editor/browser and **Print** → **Save as PDF**.

Before you start — what's already set up, and what to know

1. **The Firebase project already exists (skopeo-prod).** You do **not** need to create one. The web config values are in Step 1 below; the original "create a project" steps are kept in [Appendix A](#) only for reference.
2. **No FIREBASE_PROJECT_ID override is needed.** The API defaults `firebase.projectId` to `skopeo-prod` (its built-in default), which is exactly this project — so the web's `VITE_FIREBASE_PROJECT_ID` and the API's project already match. (If you ever point at a *different* Firebase project, set `FIREBASE_PROJECT_ID` on the API, or every login returns 401.)
3. **Facebook requires one operator step.** Facebook sign-up/login is implemented in the web UI, but for it to work at runtime the **Facebook provider must be enabled in the Firebase console** (needs a Facebook app's App ID + secret from developers.facebook.com). Until then, Google and email/password work; the Facebook button will error.
4. **The web UI is not containerized.** Locally it runs via the Vite dev server; in production it is **Firebase Hosting** (a static SPA) — *not* Docker and *not* a server/VM. So there is nothing Docker-shaped to simulate for the frontend.

How the local pieces connect

```
Browser (http://localhost:5173, Vite dev server)
|   Firebase JS SDK —————> Firebase Auth (Google)   ← returns signed ID token
|   /api/* requests —(Vite proxy)—> http://localhost:8080
▼
API container (:8080) —————> PostgreSQL container (:5432)
    verifies the ID token against Google's public JWKS
```

Prerequisites checklist

- ☐ Docker Desktop (or Docker Engine) running — `docker --version`

- ☐ Docker Compose v2 — `docker compose version`
- ☐ Node.js 20+ and npm — `node --version`
- ☐ A Google account (for the Google sign-in test)
- ☐ (For Test 3) the Facebook provider enabled in the Firebase console
- ☐ Repo cloned and you're at its root — `cd /path/to/skopeo`
- ☐ Ports free: **5173** (web), **8080** (API), **5432** (Postgres)

Part 1 — Local setup (Docker DB + API, web dev server)

Step 1 — Firebase config (already provisioned)

The project `skopeo-prod` is already set up. Just confirm/verify the values below in the Firebase console if anything looks off. These are **public client values** (safe to expose in the browser).

- ☐ Project ID: `skopeo-prod`
- ☐ Auth providers enabled (Build → Authentication → Sign-in method):
 - ☐ **Email/Password**
 - ☐ **Google**
 - ☐ **Facebook** — required only for Test 3 (see note 3 above)
- ☐ Web config:
 - `apiKey`: `AIzaSyCwx5JCXhbPSHjsXzQfSoWRUAJ2QK9mtzM`
 - `authDomain`: `skopeo-prod.firebaseio.com`
 - `projectId`: `skopeo-prod`
 - `appId`: `1:680245910471:web:857e0eed4c40d54a0d57e2`

`localhost` is an authorized domain by default, so the Google/Facebook popups work locally with no extra configuration.

Step 2 — Run PostgreSQL + API in Docker

`docker-compose.yml` builds and runs both the database (`postgres`, `:5432`) and the API (`skopeo`, `:8080`). The API already defaults `FIREBASE_PROJECT_ID` to `skopeo-prod` — **this project** — so there is nothing to override.

- ☐ 2.1 Build and start:

```
docker compose up -d --build
```

- ☐ 2.2 Watch the API come up (look for "Flyway migrations complete" and "Responding at 0.0.0.0:8080"):

```
docker compose logs -f skopeo
```

- ☐ 2.3 Health check (expect HTTP 200):

```
curl -i http://localhost:8080/health
```

- ☐ 2.4 (Optional) Database GUI:

```
docker compose --profile tools up -d pgadmin  
# http://localhost:5050 - login admin@skopeo.com / admin
```

(Only if you ever switch to a different Firebase project) create `docker-compose.override.yml` at the repo root with `services: { skopeo: { environment: { FIREBASE_PROJECT_ID: "your-id" } } }`. Not needed for `skopeo-prod`.

Step 3 — Run the web UI (dev server)

The web UI runs as the Vite dev server, which proxies `/api` to the API container. (Production is Firebase Hosting, not Docker — see note 4 and Part 2.)

- ☐ 3.1 Install deps and create the env file:

```
cd web  
npm install  
cp .env.example .env.local
```

- ☐ 3.2 Edit `web/.env.local` with the Firebase config from Step 1. **Leave** `VITE_API_BASE_URL` **empty** so it uses the dev proxy to `:8080`:

```
VITE_FIREBASE_API_KEY=AIzaSyCwx5JCXhbPSHjsXzQfSoWRUAJ2QK9mtzM  
VITE_FIREBASE_AUTH_DOMAIN=skopeo-prod.firebaseio.com  
VITE_FIREBASE_PROJECT_ID=skopeo-prod  
VITE_FIREBASE_APP_ID=1:680245910471:web:857e0eed4c40d54a0d57e2  
VITE_API_BASE_URL=
```

- ☐ 3.3 Start the dev server:

```
npm run dev      # serves http://localhost:5173
```

- ☐ 3.4 Open <http://localhost:5173> — you should see the sign-up / login screen.

Optional — mirror the production static build. Production serves the built bundle from Firebase Hosting, not the dev server. To exercise the built artifact locally: `npm run build && npm run preview`. The preview server has **no /api proxy**, so set `VITE_API_BASE_URL=http://localhost:8080` before building and add that origin to the API's CORS allow-list (`Application.configureCORS()`). For ordinary functional testing the dev server (3.3) is simpler and recommended.

Part 1b — Sign-up tests

For each test, sign up, then observe the dashboard. A brand-new account gets only the `PLAYER` capability, so the dashboard shows **only the Profile tab** with a "Pending assessment" rating — that is the expected result. (After sign-up you are auto-signed-in; the `/login` page is exercised separately in **Part 1c.**)

Test 1 — Sign up with Google

- ☐ Open <http://localhost:5173> → **Sign up**.
- ☐ Fill **Sex** and **Date of birth** (both required).
- ☐ Click **Continue with Google** → choose a Google account in the popup → land on the dashboard.

Expected: Profile tab only; name + email; a `PLAYER` badge; "Pending assessment" rating card.

Result: ☐ Pass ☐ Fail — notes: _____

Test 2 — Sign up manually (email + password)

- ☐ Sign out (if signed in) → **Sign up** → enter email, password, **Sex**, **Date of birth** → submit.

Expected: identical dashboard to Test 1. The stored email shows as unverified/PENDING for password accounts — fine for this test.

Result: ☐ Pass ☐ Fail — notes: _____

Test 3 — Sign up with Facebook

Prerequisite: the **Facebook provider must be enabled** in the Firebase console (note 3).

- ☐ Sign out → **Sign up** → fill **Sex** and **Date of birth** (required before the popup).
- ☐ Click **Continue with Facebook** → authenticate in the Facebook popup → land on the dashboard.

Expected: identical dashboard to Tests 1 & 2; the profile's identity provider is Facebook. If you reuse an email already registered via Google, expect the friendly error *"An account already exists with this email. Sign in with the original method."*

Result: ☐ Pass ☐ Fail ☐ Blocked (Facebook provider not enabled) — notes: _____

Part 1c — Login tests (returning users)

After sign-up you're already authenticated, so login is for a **fresh session**. For each method:

- ☐ Sign out, then visit <http://localhost:5173/login> (or let `RequireAuth` redirect you there).
- ☐ **Google:** click **Continue with Google** → back to the dashboard.
- ☐ **Email/password:** enter the email + password from Test 2 → **Sign in** → dashboard.
- ☐ **Facebook:** click **Continue with Facebook** → dashboard (*requires the provider enabled*).

Expected: each returns you to your existing profile (same data, no duplicate account — sign-in is idempotent on the Firebase `uid`).

Result: ☐ Google ☐ Manual ☐ Facebook — notes: _____

Optional — see the richer dashboards (Matches / Research / Admin)

Fresh `PLAYER` accounts can't show these tabs, and granting roles requires an existing admin. Bootstrap your first account as an administrator (after signing up at least once):

```
docker compose exec postgres psql -U postgres -d SkopeoDb -c \  
"INSERT INTO user_capabilities (user_id, capability)  
SELECT id, 'ADMINISTRATOR' FROM users LIMIT 1;"
```

- ☐ Refresh the dashboard → **Admin** and **Matches/Research** tabs now appear.
- ☐ As admin you can assign a starting rating, grant `HOST` to other test users, create match fixtures, and run rating calculations.

`LIMIT 1` grabs whichever user row exists first. With several test users, target a specific one by joining through `contact_information` on the email.

Teardown / reset

```
# stop the web dev server: Ctrl-C in its terminal
docker compose down          # stop API + DB (keeps data volume)
docker compose down -v       # stop AND wipe the database (fresh start next time)
```

Part 2 — Deploying the three components to GCP

Target architecture (see `DEPLOYMENT_GCP.md` for full detail):

Component	GCP service	Container?	Notes
PostgreSQL	Cloud SQL	managed	<code>db-f1-micro</code> , DB <code>SkopeoDb</code> , password in Secret Manager
Ktor API	Cloud Run	✓ Docker	built from the repo <code>Dockerfile</code> , scales to zero
React web	Firebase Hosting	✗ static	SPA from <code>web/dist</code> , deployed by <code>deploy-web.yml</code>

The web UI is a static SPA on Firebase Hosting — there is no web container or server in production. So no Docker simulation is needed for the frontend; the only prod-vs-local difference is that the deployed web reaches the API via `VITE_API_BASE_URL` (the Cloud Run URL) rather than the Vite dev proxy.

Deployment sequence

- ☐ **2.1 Project & APIs** — `gcloud projects create ...` (or use `skopeo-prod`), enable `run/sqladmin/secretmanager/cloudbuild/artifactregistry`.
- ☐ **2.2 Database (Cloud SQL)** — create the instance + `SkopeoDb` + user; store the password in Secret Manager. Note the private IP.
- ☐ **2.3 API (Cloud Run)** — deploy from source, pointing at the DB and Firebase project:

```
gcloud run deploy skopeo \
  --source . --allow-unauthenticated \
  --set-env-vars="DATABASE_URL=jdbc:postgresql://<PRIVATE_IP>:5432/SkopeoDb,DATABASE_USER=skopeo" \
  --set-secrets="DATABASE_PASSWORD=skopeo-db-password:latest"
```

`FIREBASE_PROJECT_ID=skopeo-prod` matches the API default, so it's optional — included for explicitness so the project stays pinned even if the default changes. Record the Cloud Run URL.

- ☐ **2.4 Web (Firebase Hosting)** — set GitHub repo **Variables** `VITE_FIREBASE_*` and `VITE_API_BASE_URL` (= the Cloud Run URL), plus the **Secret** `FIREBASE_SERVICE_ACCOUNT` ; then push

to `main` (or run `deploy-web.yml`). Firebase serves `web/dist` over its CDN.

Gaps to close before GCP works

- ☐ **CORS:** `Application.configureCORS()` allows only `localhost:5173`. Add the Firebase Hosting origin (`https://<project>.web.app`) and redeploy the API, or the browser blocks every call.
 - ☐ **No API deploy workflow:** there is no `deploy-api.yml`; Cloud Run deploys are manual `gcloud run deploy`.
 - ☐ **Authorized domains:** add your `*.web.app` / custom domain under Firebase Auth → Settings → Authorized domains, or Google/Facebook sign-in fails in production.
-

Troubleshooting

Symptom	Likely cause	Fix
Sign-up/login returns 401	API/web project IDs differ	Both should be <code>skopeo-prod</code> ; only an issue if you switched projects without setting <code>FIREBASE_PROJECT_ID</code> on the API
Facebook button errors / "provider disabled"	Facebook not enabled in Firebase	Enable the Facebook provider (App ID + secret) in the Firebase console
"account exists with different credential"	Email already registered via another provider	Sign in with the original method; expected Firebase behavior
Popup: "auth domain not authorized"	Domain not authorized	Add it under Firebase Auth → Settings → Authorized domains (<code>localhost</code> is there by default)
Web loads but API calls fail / CORS	Wrong base URL or proxy not used	Keep <code>VITE_API_BASE_URL</code> empty for the dev server; confirm API is up at <code>:8080</code>
<code>docker compose up</code> fails on port in use	5432/8080 taken	Stop the conflicting service or change the published port
Dashboard shows only Profile tab	Account has only <code>PLAYER</code>	Expected; bootstrap an admin (see "Optional")

Appendix A — Firebase project setup (reference)

You do **not** need this — `skopeo-prod` already exists. Kept only for recreating a project from scratch (e.g. a separate `skopeo-dev`).

1. <https://console.firebase.google.com> → **Add project**; note the **Project ID**.
 2. **Build** → **Authentication** → **Get started**.
 3. **Sign-in method** → enable **Email/Password**, **Google** (pick a support email), and **Facebook** (requires a Facebook app's App ID + secret from <https://developers.facebook.com>; register the OAuth redirect URI it shows).
 4. **Project settings** (⚙️) → **General** → **Your apps** → **Web (</>)** → register an app and copy the `apiKey`, `authDomain`, `projectId`, `appId` into `web/.env.local` (and the GitHub repo variables for deploys). The API only needs the project ID via `FIREBASE_PROJECT_ID`.
 5. Add any non-localhost domains under **Authentication** → **Settings** → **Authorized domains**.
-

Generated as a manual testing runbook for the Skopeo project.